

Notes Pack: Section 5.2 fi-model — The Model: Feature-Composed Item Factors

Placed protocol entries (in the order their \vault{} markers appear in 06_fi_protomf.tex \$fi-model), then their one-hop linked entries. Generated 2026-08-21; working material, not thesis text.

One-hop links from these entries ([[decisions]], [[paper-understanding]], [[explanations]], [[evaluation]], [[thesis-writing]], [[phase-2]], [[phase-4]]) are all hub/tag notes — skipped per the pack convention; no dated one-hop entries exist for this section.

[placed] 2026-06-16_1715_feature-injection-design-fork

Design fork: feature injection via prediction vs prototype-space prior

Identified a design fork for feature-aware ProtoMF from S7 Eq 25 (Chen et al. 2020, §4.1.1), which has two slots for attributes: the likelihood (additive terms $\alpha/\beta/\gamma$ entering the prediction) and the priors $p(W|X)p(H|Y)$ on latent factors. Mapped to two candidate routes: **(A)** features feed into the entity representation/score directly; **(B)** features act as a prior/regularizer shaping the prototype space without touching the score — natural fit since ProtoMF already has structured-space regularizers (R_{proto} , R_{batch}). S7 flags route (B) as “not yet observed” in the linear category, making it a **citable novelty angle**. Not mutually exclusive. Logged to design-candidate scratchpad for a future design cycle; decision deferred. Relates to [[decisions]] and the existing scratchpad entry on attaching feature profiles to CF prototypes.

[[decisions]] [[paper-understanding]] [[phase-4]] [[phase-2]]

[placed] 2026-07-11_1526_facet-a-claim-boundary-coldstart-2x2

Facet A claim boundary + cold-start 2×2 (S0.2 scrutiny)

Two structural insights from the S0.2 side-reasoning step (scrutiny dossier Master/docs/scrutiny/s0_foundation §S0.2), both following from the double-tie score decomposing into **Block I** (user affinities · item-prototype sims) and **Block U** (user-prototype sims · item projection), with all four candidates touching only the item branch. **(1) Facet A claim boundary:** the candidates ground *what item prototypes mean* (intrinsic, feature-grounded vocabulary — $R_2/R_3/R_4$), but the personalization weights $\hat{u}_k$ stay learned CF — claims must say “explanations are stated in intrinsic feature vocabulary,” never “the model’s reasoning is feature-transparent.” Binding methodological rule adopted: every per-recommendation read-out must report the **score share of Block I vs Block U** on real checkpoints (SC.5/SC.8), else intrinsicness could describe a minority of the score. **(2) Cold-start 2×2:** in a bilinear score one warm side suffices — warm-user × cold-item works with item features alone (learned taste meets attribute-derived placement in the shared prototype space); cold users are structurally out of reach for item-side designs, would need user-side features, and don’t exist in our split anyway. Thesis cold-start claims are scoped to cold items + warm-but-thin users; H&M is an item-cold domain (catalog churn), so the scoping matches the domain. Follow-up design ideas (dual-side attribute-aware candidate; 3-scenario cold testbed) captured in design_candidates/scratchpad.md.

[placed] 2026-07-13_2113_why-f-not-attribute-in-model-names-mechanism-vs-contribution

Why the *f* in *fI*-/*fU*-ProtoMF stays “feature”, not “attribute” — mechanism vs contribution naming

(Verbatim-capture entry — terminology justification from the thesis-outline session, user requested protocol after adversarial check. Complements the attribute/feature convention: “attribute” = human-readable catalogue property (semantics, prose); “feature” = encoded, model-facing input row (mechanism).)

DIRECTIVE (Matteo, 2026-07-13): in the thesis this is a **SINGLE SENTENCE** — one line in the definitional/terminology passage justifying the *f* in the model names, not a paragraph or section. The entry below preserves the full argument so that sentence can be written (and defended if asked); the thesis gets only the sentence.

The justification (Matteo, near-literal)

fI-ProtoMF — meaning *feature*-Item ProtoMF — is a better name than *attribute*-Item ProtoMF, “because we can feed whatever we want in there. If we use the ID version, a database ID is arguable whether it is an attribute of the item. We could also feed something else in there entirely — something correcting bias in some way. **The model does not know and does not care.**”

Why it holds (checked together)

Names should describe contracts, not intentions. The composition mechanism’s contract is: sum whatever encoded input rows arrive ($q = \Sigma$ rows). “Feature” in the pinned model-facing sense describes exactly that contract. “Attribute” would over-promise — it would claim the inputs are semantically meaningful catalogue properties, which the ID row already violates today (a database handle is not a catalogue property), and which any future engineered row (e.g. a bias-correcting input) would violate further.

The altitude resolution: mechanism = *feature-composed* (hence the *f* in model names, and the code’s *feature_** naming needs no apology); contribution = *attribute-aware* (hence Part I’s title and §1.3). *Choosing* attribute-derived rows is what turns the agnostic mechanism into an interpretable model — the interpretability lives in the choice of rows, not in the summation.

Retroactive confirmation from dc02: the codebase already implicitly follows this rule — dc02 is *attr_item_proto*, and there “attr” is correct because that design anchors prototypes in attribute space *by construction* (enforced grounding); the semantics are baked into the mechanism, not chosen by the operator. Rule made explicit: **attr = semantics enforced by the mechanism; feature = semantics-agnostic composition.**

One honest caveat for the prose: the agnosticism is *architectural*, not behavioral — training shapes what each row comes to mean. The model doesn’t care what you feed it, but it decides what to make of it.

Future-proofing side effect: if an engineered non-semantic row is ever added (hidden-effects work flirts with such ideas), the naming scheme accommodates it without a retcon.